DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

PROJECT REPORT

On

Traffic Monitoring and Statistics Collector using Software Defined Networking (SDN)

Submitted by

Dhanush Rao
SRN: PES2UG24CS153

Course: Computer Networks

PES UNIVERSITY
Academic Year: 2025–26

Github link: https://github.com/dhanushrao-max/Traffic-Monitoring-and-Statistics-Collector

Abstract

Software Defined Networking (SDN) is a transformative approach to computer networking that separates the control plane from the data plane. In traditional networks, each device independently makes forwarding decisions. SDN moves this intelligence to a centralized software controller, which communicates with the underlying hardware using standardized protocols such as OpenFlow.

This project implements an SDN-based Traffic Monitoring and Statistics Collector using the POX controller and Mininet network emulator. The system performs MAC address learning, dynamic packet forwarding, and selective traffic blocking (firewall). It also collects real-time flow statistics — specifically packet counts and byte counts — from OpenFlow switches at regular intervals.

The collected data is stored in a CSV file and visualized using a Python-based graph plotting script. The project demonstrates how SDN enables centralized and programmable network management, making it easy to monitor traffic, enforce policies, and gain insights into network behavior.

1. Introduction

1.1 Traditional Networking vs SDN

In conventional computer networks, each network device — such as routers and switches — contains both the control logic and the data forwarding hardware. This tight integration means that any changes to network policies (like routing decisions, access control, or QoS) must be configured individually on each device. Managing such distributed systems is complex, error-prone, and slow to respond to changing network conditions.

Software Defined Networking (SDN) revolutionizes this model. In an SDN environment, a central software controller manages the overall network behavior. The controller communicates with the hardware switches using the OpenFlow protocol, instructing them on how to forward packets. The switches themselves are simple and only perform data forwarding — the intelligence lies entirely in the controller.

1.2 Need for Centralized Control

As networks grow larger and more dynamic — especially in cloud computing environments and data centers — the ability to manage them centrally becomes essential. A centralized controller provides a global view of the entire network, enabling better resource utilization, faster policy changes, and simplified troubleshooting. Administrators can program new rules from a single point rather than logging into dozens of devices.

1.3 Importance of Traffic Monitoring

Traffic monitoring is a fundamental aspect of network management. By tracking the number of packets and bytes flowing through a network, administrators can detect congestion, identify unusual traffic patterns, enforce usage policies, and plan for capacity upgrades. In SDN, traffic monitoring becomes significantly easier because the controller can request flow statistics directly from the switches and aggregate this information for analysis.

This project leverages these SDN capabilities to build a complete traffic monitoring system that collects, stores, and visualizes network statistics in real time.

2. Objectives

The primary objectives of this project are as follows:

- Implement an SDN controller using the POX framework that can manage an OpenFlow-based virtual network.
- Enable dynamic MAC address learning and intelligent packet forwarding within the SDN environment.
- Implement a simple firewall rule to block specific traffic flows between defined hosts.
- Collect flow-level statistics — including packet count and byte count — from OpenFlow switches at periodic intervals.
- Store the collected statistics in a structured CSV file for record-keeping and analysis.
- Visualize the traffic trends over time using a Python graph plotting script based on the Matplotlib library.
- Demonstrate the practical value of SDN in enabling programmable, centralized, and monitorable networks.

3. System Architecture

The system consists of three major components that interact with each other through standard protocols. These components are Mininet, the POX Controller, and OpenFlow-enabled virtual switches.

3.1 Mininet

Mininet is a network emulator that creates a realistic virtual network on a single machine. It instantiates virtual hosts, switches, and links. These virtual devices behave like real network hardware and can be programmed using standard networking protocols. Mininet is the platform on which the test network topology is created for this project.

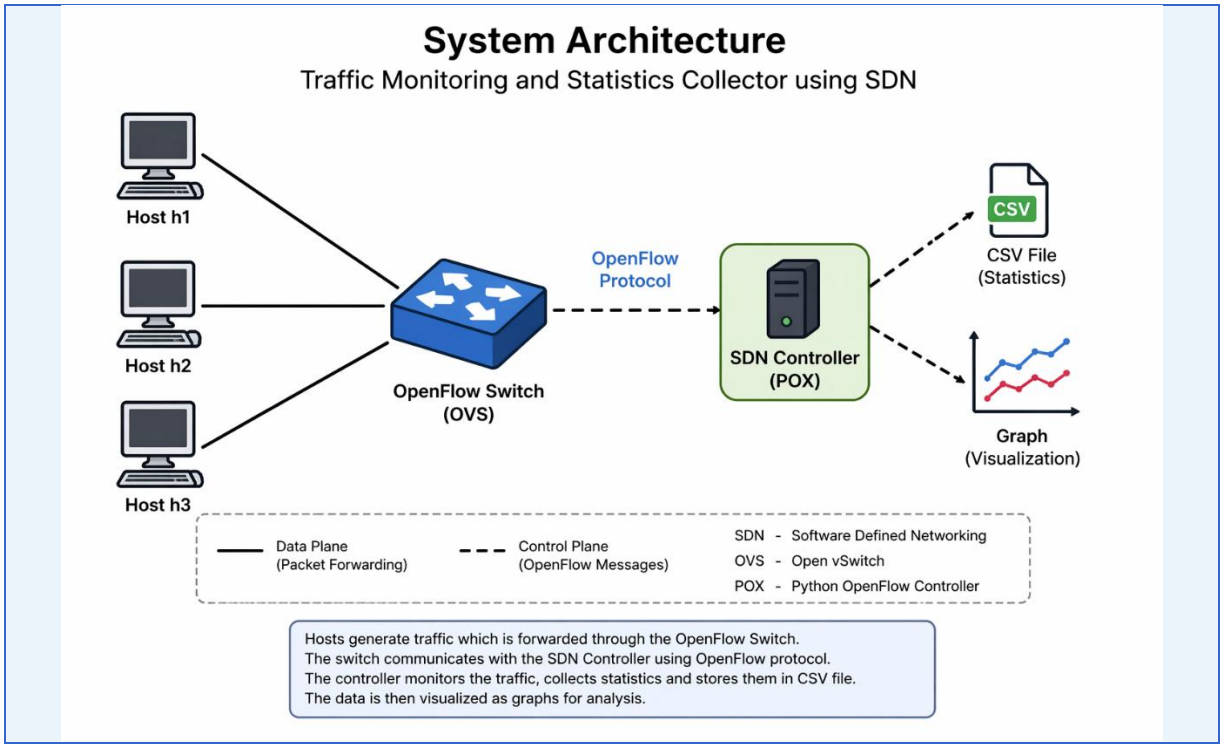
3.2 POX Controller

POX is an open-source Python-based SDN controller. It acts as the brain of the network. When a new packet arrives at a switch that does not know how to handle it, the switch forwards that packet to the POX controller. The controller then decides how the switch should handle it and installs appropriate flow rules into the switch for future packets.

3.3 OpenFlow Switch

OpenFlow switches are forwarding devices that implement the OpenFlow protocol. They maintain a flow table — a set of rules that determine how each packet should be forwarded. When a packet arrives and matches a rule, it is forwarded accordingly. When no rule matches, the switch sends a PacketIn event to the controller for instructions.

3.4 Architecture Diagram



The diagram above illustrates the relationship between the hosts, the OpenFlow switch, and the POX controller. All control decisions flow through the controller, while actual data traffic travels directly through the switch.

4. Technologies Used

4.1 Python

Python is the primary programming language used throughout this project. Both the POX controller module (traffic_monitor.py) and the graph plotting script (plot_graph.py) are written in Python. Python's clear syntax and rich library ecosystem make it ideal for rapid network programming and data analysis.

4.2 POX Controller

POX is a Python-based SDN controller developed at UC Berkeley. It provides event-driven APIs for handling OpenFlow events such as PacketIn (new packet received), ConnectionUp (switch connected), and FlowStatsReceived (statistics returned). POX includes built-in support for OpenFlow message construction, MAC learning, and timer scheduling.

4.3 Mininet

Mininet is a widely-used network emulation platform developed at Stanford University. It creates virtual networks using Linux network namespaces and virtual Ethernet pairs. Mininet networks are realistic enough to run actual networking software and protocols, making it ideal for SDN research and education.

4.4 OpenFlow Protocol

OpenFlow is the standardized communication protocol used between the SDN controller and the OpenFlow-enabled switches. It defines the format of messages such as FlowMod (install a rule), PacketIn (forward unknown packet to controller), PacketOut (send a packet from controller), and StatsRequest/StatsReply (request and receive flow statistics).

4.5 Matplotlib

Matplotlib is a Python library for creating static, animated, and interactive visualizations. In this project, it is used in plot_graph.py to read the CSV file of traffic statistics and plot line graphs showing how packet counts and byte counts change over time.

5. Methodology

This section explains in detail the step-by-step process through which the SDN traffic monitoring system was designed and implemented.

5.1 Network Creation Using Mininet

The virtual network is created using Mininet. A simple topology consisting of two hosts (h1 and h2) connected to a single OpenFlow switch (s1) is used. This switch is configured to connect to the POX controller running locally. The Mininet topology is started with the following command:

```
$ sudo mn --controller=remote,ip=127.0.0.1 --mac --topo single,2
```

The --mac flag assigns simple MAC addresses (00:00:00:00:00:01 for h1 and 00:00:00:00:00:02 for h2) to the hosts, making them easy to identify in the controller code.

5.2 Controller–Switch Interaction: PacketIn and FlowMod

When a host sends its first packet, the switch checks its flow table. Since no rules exist initially, the switch encapsulates the packet in a PacketIn message and sends it to the POX controller. The controller's _handle_PacketIn function is invoked, which examines the packet header and decides the forwarding action.

Once the correct output port is determined, the controller sends an ofp_flow_mod (Flow Modification) message back to the switch. This message installs a new rule in the switch's flow table so that future packets with the same header can be forwarded directly by the switch without involving the controller again. This mechanism greatly improves performance after the initial rule installation.

5.3 MAC Learning and Forwarding Logic

The controller maintains a dictionary called mac_to_port that maps each switch (identified by its datapath ID, or dpid) to its known MAC address-to-port mappings. When a packet arrives, the controller learns the source MAC address and records which port it arrived on. Then, when forwarding, the controller checks whether the destination MAC address is already known.

If the destination MAC is known, the packet is forwarded to the specific port. If it is unknown, the packet is flooded out all ports (OFPP_FLOOD), ensuring delivery at the cost of temporary network-wide broadcasting.

5.4 Firewall Implementation

A simple software-defined firewall rule is implemented inside the _handle_PacketIn function. When the source MAC address is 00:00:00:00:00:01 (host h1) and the destination is 00:00:00:00:00:02 (host h2), the packet is silently dropped by returning early from the function. This prevents the flow rule from being installed and stops all traffic between these two hosts.

This approach demonstrates how SDN enables policy enforcement through centralized software logic, without any physical reconfiguration of the network hardware.

5.5 Flow Statistics Collection

The controller registers a handler for the FlowStatsReceived event. When this event fires, the controller receives a list of flow statistics from the switch. Each stat object contains packet_count (the number of packets matched by the rule) and byte_count (the total bytes forwarded). These values are printed to the console and written to a CSV file with a timestamp.

5.6 Periodic Monitoring Using Timer

A recurring timer is configured using POX's built-in Timer class. Every 5 seconds, the request_stats() function is called, which sends an ofp_stats_request message to all connected switches. This ensures that the controller continuously pulls fresh statistics from the network, enabling near real-time monitoring.

5.7 CSV File Generation

The traffic statistics are stored in a file called traffic_stats.csv. This file is created automatically if it does not already exist. Each row written to the file contains three values: the current Unix timestamp (from time.time()), the packet count, and the byte count. This structured format allows easy processing with standard tools.

5.8 Graph Plotting

The plot_graph.py script reads the traffic_stats.csv file and uses Matplotlib to plot two line graphs — one for packets and one for bytes — both plotted against time. The graph provides a clear visual representation of how traffic has evolved over the monitoring period, helping network administrators identify spikes or anomalies.

6. Implementation Details

6.1 traffic_monitor.py — Controller Logic

This file is the heart of the project. It implements a POX controller module that handles all network events. The key components are:

- mac_to_port: A nested dictionary that records which port each MAC address can be reached through, on each switch.
- _handle_ConnectionUp: Called when a switch connects to the controller. Logs the connection for debugging purposes.
- _handle_PacketIn: The main event handler. It performs MAC learning, applies the firewall rule, determines the output port, installs a flow rule, and sends the packet out.
- _handle_FlowStatsReceived: Processes flow statistics received from the switch and appends them to the CSV file.

- request_stats: Sends a statistics request to all connected switches to pull current flow counters.
- launch: The POX entry point that registers all event listeners and starts the recurring timer.

Below is the key section of traffic_monitor.py showing the PacketIn handler:

```
def _handle_PacketIn(event):
    packet = event.parsed
    dpid = event.connection.dpid
    mac_to_port.setdefault(dpid, {})
    mac_to_port[dpid][packet.src] = event.port

    # Firewall: block h1 → h2
    if packet.src == '00:00:00:00:00:01' and \
       packet.dst == '00:00:00:00:00:02':
        return

    out_port = mac_to_port[dpid].get(packet.dst, of.OFPP_FLOOD)
    # Install flow rule and send packet ...
```

6.2 plot_graph.py — Visualization

This script reads traffic_stats.csv and uses Matplotlib to generate a dual-line chart. The X-axis shows the time values (Unix timestamps) and the Y-axis shows the count of packets and bytes. The chart includes a legend, axis labels, and a title. The visualization is rendered in a pop-up window using plt.show().

```
with open('traffic_stats.csv', 'r') as f:
    reader = csv.reader(f)
    next(reader)  # skip header
    for row in reader:
        time_vals.append(float(row[0]))
        packets.append(int(row[1]))
        bytes_.append(int(row[2]))

plt.plot(time_vals, packets, label='Packets')
plt.plot(time_vals, bytes_, label='Bytes')
plt.xlabel('Time')  plt.ylabel('Count')
plt.title('Traffic Monitoring')  plt.legend()
plt.show()
```

6.3 Flow Rule Installation

Flow rules are installed using the ofp_flow_mod object from the POX OpenFlow library. A match object is derived from the incoming packet using ofp_match.from_packet(), ensuring that the rule is specific to the exact traffic pattern (source/destination MAC, input port, etc.). The action added to the flow rule directs matching packets to the determined output port. This rule is installed proactively so that the switch can handle subsequent packets autonomously.

6.4 Data Logging Process

Every time the _handle_FlowStatsReceived event is triggered (every 5 seconds), the handler iterates over all flow statistics returned. For each flow, it extracts the packet_count and byte_count, combines them with the current timestamp, and appends a new row to traffic_stats.csv. The file remains open only for the duration of each write operation to avoid data corruption.

7. Results and Observations

7.1 Behavior During Ping

When the ping command is executed between h1 and h2 in Mininet, the ARP request is the first packet that reaches the controller. The controller learns the MAC addresses and installs flow rules accordingly. Subsequent ICMP packets are forwarded by the switch directly, bypassing the controller.

When the firewall rule is active, pings from h1 to h2 fail (100% packet loss). The controller drops those packets without installing any flow rule. Communication in the reverse direction (h2 to h1) is unaffected and succeeds normally.

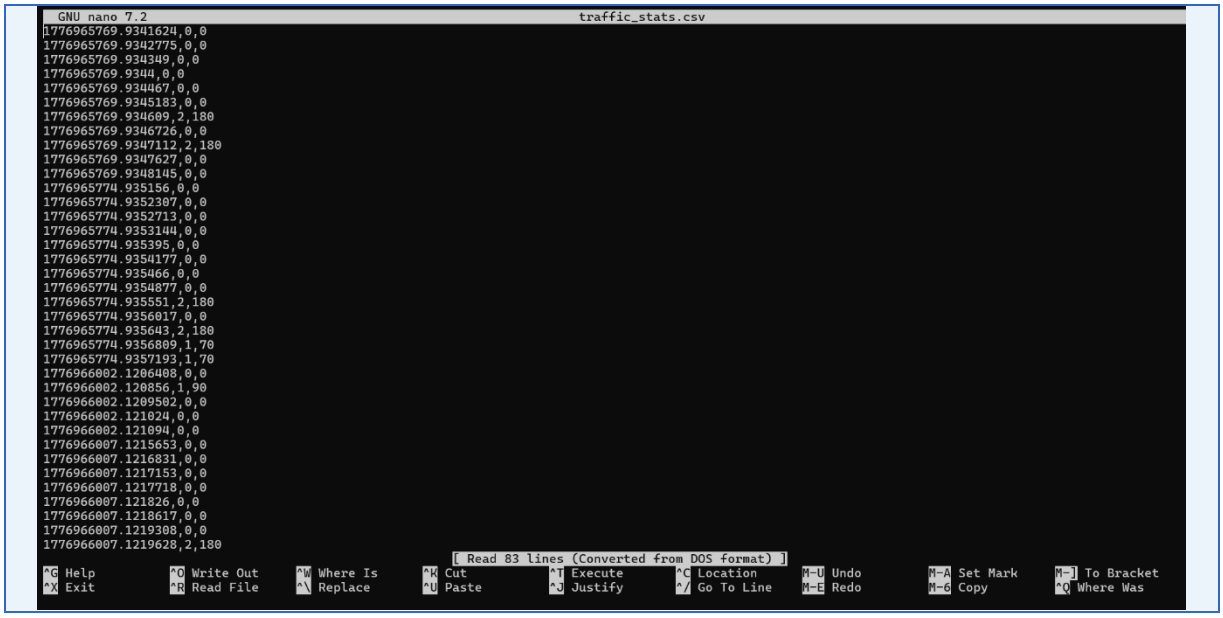
7.2 Controller Terminal Output

```
dhanushrao@Dhanush:~/pox$ ./pox.py log.level --DEBUG openflow.of_01 traffic_monitor
POX 0.7.0 (gar) / Copyright 2011-2020 James McCauley, et al.
DEBUG:core:POX 0.7.0 (gar) going up...
DEBUG:core:Running on CPython (3.12.3/Mar 3 2026 12:15:18)
DEBUG:core:Platform is Linux-6.6.87.2-microsoft-standard-WSL2-x86_64-with-glibc2.39
WARNING:version:POX requires one of the following versions of Python: 3.6 3.7 3.8 3.9
WARNING:version:You're running Python 3.12.
WARNING:version:If you run into problems, try using a supported version.
INFO:core:POX 0.7.0 (gar) is up.
DEBUG:openflow.of_01:Listening on 0.0.0.0:6633
INFO:openflow.of_01:[00-00-00-00-00-01 2] connected
INFO:traffic_monitor:Switch connected: [00-00-00-00-00-01 2]
Packets: 0 Bytes: 0
Packets: 0 Bytes: 0
Packets: 0 Bytes: 0
Packets: 0 Bytes: 0
Packets: 0 Bytes: 0
Packets: 0 Bytes: 0
Packets: 2 Bytes: 180
Packets: 0 Bytes: 0
Packets: 2 Bytes: 180
Packets: 0 Bytes: 0
Packets: 0 Bytes: 0
Packets: 0 Bytes: 0
Packets: 0 Bytes: 0
Packets: 0 Bytes: 0
Packets: 0 Bytes: 0
Packets: 0 Bytes: 0
Packets: 0 Bytes: 0
Packets: 0 Bytes: 0
Packets: 0 Bytes: 0
Packets: 2 Bytes: 180
Packets: 0 Bytes: 0
Packets: 2 Bytes: 180
Packets: 1 Bytes: 70
Packets: 1 Bytes: 70
^CINFO:core:Going down...
INFO:openflow.of_01:[00-00-00-00-00-01 2] disconnected
INFO:core:Down.
dhanushrao@Dhanush:~/pox$ sudo mn --controller=remote,ip=127.0.0.1,port=6633 --switch ovs,protocols=OpenFlow10
```

```
dhanushrao@Dhanush:~/pox$ sudo mn --controller=remote,ip=127.0.0.1,port=6633 --switch ovs,protocols=OpenFlow10
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2
*** Adding switches:
s1
*** Adding links:
(h1, s1) (h2, s1)
*** Configuring hosts
h1 h2
*** Starting controller
c0
*** Starting 1 switches
s1 ...
*** Starting CLI:
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2
h2 -> h1
*** Results: 0% dropped (2/2 received)
mininet> exit
*** Stopping 1 controllers
c0
*** Stopping 2 links
..
*** Stopping 1 switches
s1
*** Stopping 2 hosts
h1 h2
*** Done
completed in 19.061 seconds
dhanushrao@Dhanush:~/pox$
```

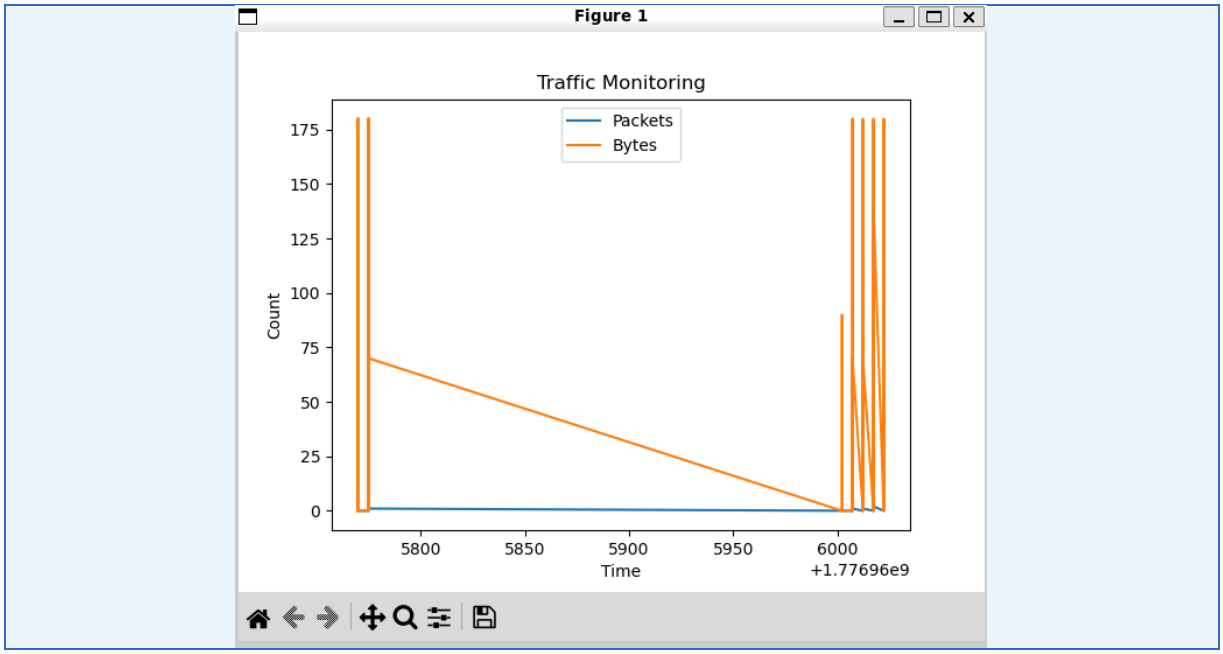
7.3 CSV File

The traffic_stats.csv file is populated with rows every 5 seconds. Each row contains the Unix timestamp, total packet count, and total byte count as reported by the switch's active flow entries. The file grows continuously as long as the controller and network are running.



7.4 Graph Output

The graph plotted by plot_graph.py shows two curves over time. The packets curve shows how many packets have been forwarded through each flow rule. The bytes curve shows the total data volume. In a typical ping test, both curves climb steadily and then plateau when the ping session ends. Spikes indicate periods of heavy traffic.



7.5 Observations

- The MAC learning mechanism correctly reduces flooding after the first packet exchange.
- The firewall rule blocks h1-to-h2 traffic effectively without disrupting other flows.
- Flow statistics are polled every 5 seconds, providing near real-time monitoring.
- The CSV file captures a full history of traffic behavior throughout the experiment.
- The graph clearly shows distinct traffic events, making it useful for human analysis.

8. Advantages

8.1 Centralized Control

The POX controller has a complete global view of the network. All policy decisions — forwarding, blocking, monitoring — are made from a single point. This eliminates the need to configure individual devices and reduces administrative overhead significantly.

8.2 Real-Time Traffic Monitoring

The periodic timer mechanism ensures that the controller collects and logs flow statistics every 5 seconds. This provides a near real-time view of network activity, enabling administrators to detect anomalies or usage spikes quickly.

8.3 Easy Policy Enforcement

Firewall rules and forwarding policies are implemented as simple Python code in the controller. Changing a rule requires only a code modification — there is no need to log into hardware devices and reconfigure access control lists (ACLs). This makes the system highly agile and policy-responsive.

8.4 Scalable Design

The architecture easily scales to larger networks. Additional hosts and switches can be added to the Mininet topology, and the controller code handles them automatically through the same event-driven logic. The statistics collection is not limited to a fixed number of flows.

9. Limitations

9.1 Simulated Environment

The project uses Mininet, which is an emulator and not a real hardware deployment. While Mininet faithfully simulates network behavior, real-world deployments may involve additional complexity such as hardware latency, physical link failures, and vendor-specific OpenFlow implementations.

9.2 Limited to Basic Flows

The current implementation handles only Layer 2 (MAC-based) flow rules. It does not perform Layer 3 IP routing or handle more complex protocols like VLAN tagging, MPLS, or multicast. The firewall rule is also simplistic — it blocks traffic based on fixed MAC addresses rather than dynamic policies.

9.3 No GUI Dashboard

The monitoring output is either printed to the terminal or saved to a CSV file. There is no live dashboard or web-based interface for visualizing traffic in real time. The Matplotlib graph requires the experiment to be completed before it can be plotted, since it reads from the CSV file after the fact.

10. Applications

10.1 Network Monitoring Systems

This project directly models the behavior of professional network monitoring tools used in enterprise environments. By extending the statistics collection and adding thresholds, the system can alert administrators when usage exceeds safe levels.

10.2 Data Centers

Modern data centers use SDN extensively to manage traffic between thousands of servers. The centralized monitoring approach implemented here is directly applicable to data center fabric management, where real-time visibility into traffic is critical for load balancing.

10.3 Traffic Analysis

Network engineers can use systems like this to analyze traffic patterns, identify bandwidth hogs, and understand communication flows between different parts of the network. The CSV data can be fed into analytics pipelines for long-term trend analysis.

10.4 Security Systems

The firewall capability demonstrated in this project can be extended to build intrusion detection systems (IDS) and dynamic access control systems. The controller can be programmed to detect suspicious traffic patterns and automatically block offending flows.

11. Future Enhancements

11.1 GUI Dashboard

A web-based dashboard could be developed using Flask or Django to display real-time traffic statistics. The dashboard could update automatically using WebSockets or polling, providing administrators with a live view of network activity without needing to inspect the terminal or CSV files manually.

11.2 Advanced Anomaly Detection

Machine learning models could be integrated to analyze traffic patterns over time and automatically detect anomalies such as DDoS attacks, port scans, or unusual bandwidth consumption. These models could be trained on historical CSV data and deployed alongside the POX controller.

11.3 Real-Time Alerts

The system could be extended to send automated alerts (via email, SMS, or Slack) when traffic exceeds predefined thresholds. This would enable rapid incident response without requiring an administrator to watch the dashboard at all times.

11.4 Integration with Cloud Systems

The SDN controller could be adapted to integrate with cloud management platforms such as OpenStack or Kubernetes networking. This would allow traffic monitoring and policy enforcement to extend into cloud-native workloads and microservices environments.

12. Conclusion

This project successfully demonstrates the power and flexibility of Software Defined Networking for traffic monitoring and statistics collection. By using the POX controller with Mininet, a complete and functional SDN environment was created that performs MAC learning, dynamic packet forwarding, firewall policy enforcement, and real-time flow statistics collection.

The system stores traffic data in a structured CSV format, which is then visualized using Matplotlib to provide meaningful graphical insights. The implementation clearly shows how SDN's centralized architecture simplifies network management tasks that would traditionally require complex, distributed configuration.

The project reinforces key concepts of computer networking — including the OpenFlow protocol, flow tables, and packet processing pipelines — while also providing hands-on experience with practical tools like Python, POX, and Mininet. The results validate that SDN is a viable and powerful paradigm for building intelligent, programmable, and observable networks.

Future work can significantly extend this foundation by adding a live web dashboard, machine learning-based anomaly detection, and cloud integration, making the system suitable for enterprise-grade deployment.

13. References

1. Mininet Project. Mininet: An Instant Virtual Network on your Laptop (or other PC). Available at: http://mininet.org/
2. POX Controller Documentation. NOX/POX Network Controller. Available at: https://noxrepo.github.io/pox-doc/html/
3. Open Networking Foundation. OpenFlow Switch Specification Version 1.0.0. Available at: https://opennetworking.org/wp-content/uploads/2013/04/openflow-spec-v1.0.0.pdf
4. Lantz, B., Heller, B., & McKeown, N. (2010). A Network in a Laptop: Rapid Prototyping for Software-Defined Networks. Proceedings of the 9th ACM SIGCOMM Workshop on Hot Topics in Networks (HotNets-IX).
5. McKeown, N., et al. (2008). OpenFlow: Enabling Innovation in Campus Networks. ACM SIGCOMM Computer Communication Review, 38(2), 69–74.
6. Matplotlib Development Team. Matplotlib: Visualization with Python. Available at: https://matplotlib.org/
7. Python Software Foundation. Python 3 Documentation. Available at: https://docs.python.org/3/